ГУАП

Кафедра №14

ОТЧЁТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

старший преподаватель		А.Ю. Петров
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ №1

Разработка класс Keeper, который реализует контейнер для хранения и обработки объектов

по курсу: ТЕХНОЛОГИЯ ПРОГРАММИРОВАНИЯ

РАБОТУ ВЫПОЛНИЛ

Студент гр. № 7441	12.10.2026	Д.В. Усачев
	подпись, дата	инициалы, фамилия

Санкт-Петербург 2026

СОДЕРЖАНИЕ

1 Постановка задачи 3
2 Формализация задачи 4
3 Таблица описания классов 6
4 Исходный код 8
5 Результаты работы программы 20
6 Выводы 23

1 Постановка задачи

Необходимо разработать класс Keeper, который реализует контейнер для хранения и обработки объектов. Класс Keeper должен обеспечивать следующие функции:

- добавление и удаление производных объектов абстрактного класса Base;
- полное сохранение себя в файле;
- полное восстановление себя из файла.

Вариант 1. Класс ВУЗ хранит данные по студентам, преподавателям и административному персоналу. Для каждого студента определено: ФИО, группа, наименование специальности, текущий курс, средний балл. Для каждого преподавателя определено: ФИО; перечислены группы, которые ведет; предметы, которые преподает. Для административного персонала определено: ФИО, должность, телефон, область ответственности.

Требования к реализации:

- работа с динамическим выделением памяти;
- запрет на использование STL-контейнеров;
- полное пользовательское меню: все параметры вводятся с клавиатуры, значения в исходном коде явно не задаются;
- доступ пользователя к каждому из объектов-наследников: добавление, удаление, вывод на экран, изменение данных;
- конструкторы (с параметрами, без параметров, копирования) и деструктор, каждый вызов которых сопровождается сообщением пользователю;
- переопределение операторов;
- методы просмотра и установки значений (get и set);
- собственные исключения с текстом, поясняющим возникшую ситуацию;
- разделение на файлы .h и .cpp для каждого класса, функция main в отдельном .cpp файле.

2 Формализация задачи

Формализация — отображение результатов мышления в точных понятиях и утверждениях. Ниже предметная область «ВУЗ» представлена в виде набора сущностей, их свойств и операций над ними.

2.1 Сущности предметной области

ВУЗ хранит три вида объектов: студентов, преподавателей и административный персонал. У всех объектов есть общее свойство — ФИО, поэтому оно вынесено в абстрактный базовый класс Base. Остальные свойства определяются классами-наследниками (таблица 1).

Таблица 1 — Сущности и их свойства

Сущность	Свойство	Тип данных
Base (общее)	ФИО	строка
Student	Группа	строка
Student	Наименование специальности	строка
Student	Текущий курс (от 1 до 6)	целое число
Student	Средний балл (от 0 до 5)	вещественное число
Teacher	Группы, которые ведет	динамический массив строк
Teacher	Предметы, которые преподает	динамический массив строк
Admin	Должность	строка
Admin	Телефон	строка (цифры и символы + - ( ) и пробел)
Admin	Область ответственности	строка

Таблица 1 показывает, какие данные хранит каждая сущность. Для преподавателя количество групп и предметов заранее неизвестно, поэтому они хранятся в динамических массивах.

2.2 Операции над данными

Пользователь должен иметь возможность выполнять следующие операции:

– добавить студента, преподавателя или сотрудника администрации;
– вывести на экран все объекты или один объект по номеру;
– изменить данные объекта по номеру;
– удалить объект по номеру;
– сохранить весь ВУЗ в файл и восстановить его из файла (имя файла вводится пользователем).

2.3 Модель решения

Для решения использованы следующие приемы объектно-ориентированного программирования:

- абстракция: класс Base объявляет чисто виртуальные методы print, input, save, load, clone, type и typeName;
- наследование: Student, Teacher и Admin открыто наследуют Base;
- полиморфизм: Keeper хранит указатели Base* и вызывает виртуальные методы, не зная конкретного типа объекта;
- композиция: Keeper владеет массивом указателей на объекты и удаляет их в деструкторе;
- обработка исключений: ошибки ввода, индекса, файла и формата данных описаны отдельными классами исключений.

2.4 Формат файла сохранения

Файл текстовый, каждое значение записано на отдельной строке. Первая строка — количество объектов. Далее для каждого объекта записывается код типа (1 — студент, 2 — преподаватель, 3 — администрация), затем поля объекта в порядке, указанном в таблице 1. Для преподавателя перед списком групп и перед списком предметов записывается их количество.

2.5 Исключительные ситуации

Таблица 2 — Исключения программы

Исключение	Когда возникает
InputException	пустая строка, нечисловой ввод, значение вне допустимого диапазона, закрытый поток ввода
IndexException	обращение к несуществующему элементу контейнера или массива
FileException	файл не удалось открыть или записать
FormatException	файл поврежден: неожиданный конец данных, неизвестный код типа, некорректное количество

В таблице 2 перечислены все исключения, которые выбрасывает программа. Каждое содержит текст на русском языке, который выводится пользователю в главном цикле меню, после чего работа продолжается.

2.6 Работа с памятью

Объекты создаются оператором new и принадлежат контейнеру Keeper. Массив указателей в Keeper удваивается при заполнении (начальная емкость — 4). Массивы групп и предметов преподавателя пересоздаются при каждом добавлении элемента. Память

освобождается в деструкторах. При загрузке из файла данные сначала читаются во временный контейнер, поэтому при ошибке текущее содержимое не теряется.

3 Таблица описания классов

Таблица 3 — Описание классов

Имя класса	Назначение	Наследование и его тип
Base	Абстрактный базовый класс: хранит ФИО, задает интерфейс вывода, ввода, сохранения, загрузки и копирования объектов	Не наследуется
Student	Студент: группа, специальность, курс, средний балл	От Base, открытое (public)
Teacher	Преподаватель: группы и предметы в динамических массивах	От Base, открытое (public)
Admin	Административный персонал: должность, телефон, область ответственности	От Base, открытое (public)
Keeper	Контейнер (ВУЗ): динамический массив указателей Base*, добавление, удаление, вывод, сохранение в файл и загрузка из файла	Не наследуется; владеет объектами Base (композиция)
AppException	Базовое исключение приложения с текстом ошибки	От std::exception, открытое (public)
InputException	Ошибки ввода данных	От AppException, открытое (public)
IndexException	Выход за границы контейнера или массива	От AppException, открытое (public)
FileException	Ошибки открытия и записи файла	От AppException, открытое (public)
FormatException	Некорректный формат файла сохранения	От AppException, открытое (public)

В таблице 3 перечислены все классы программы. Во всех классах определены конструкторы (без параметров, с параметрами, копирования), деструктор, get- и set-методы. Конструкторы и деструкторы выводят сообщение с указанием объекта и вида вызова. Иерархия связей классов показана на рисунке 1.

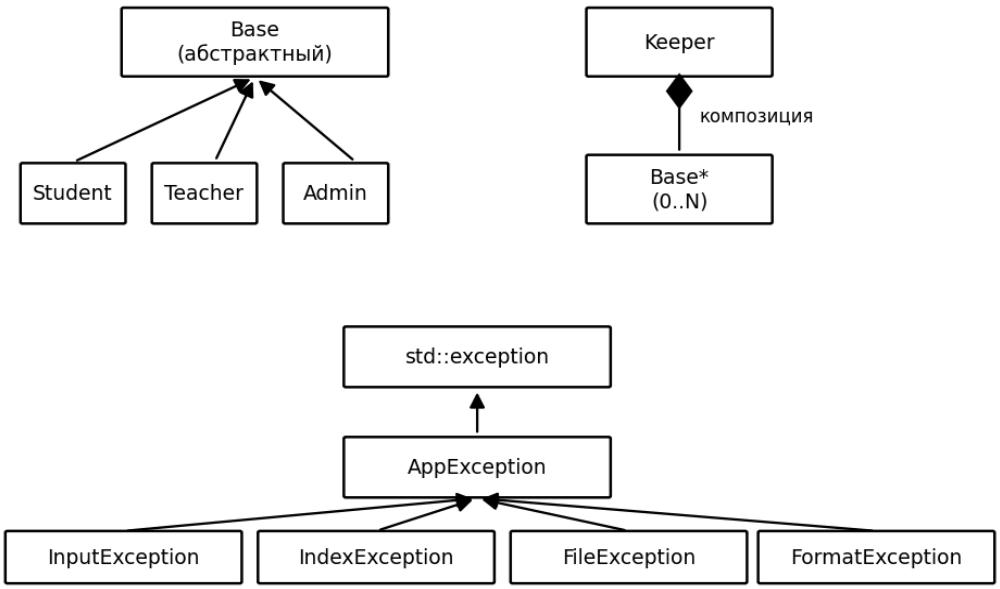


Рисунок 1 — Иерархия связей классов

На рисунке 1 показано наследование: Student, Teacher и Admin наследуются от абстрактного класса Base, а четыре класса исключений — от AppException. Ромб обозначает композицию: контейнер Keeper хранит указатели на объекты Base и отвечает за их удаление.

4 Исходный код

Ссылка на проект в системе контроля версий: [вставить ссылку на репозиторий GitHub].

Полный исходный код приведен ниже. Проект состоит из 14 файлов: для каждого класса выделены заголовочный файл (.h) с определением и файл реализации (.cpp), функция main расположена в отдельном файле main.cpp.

4.1 Файл Exceptions.h

```
#ifndef EXCEPTIONS_H
#define EXCEPTIONS_H
#include <exception>
#include <string>

// Базовое исключение приложения: хранит поясняющий текст
class AppException : public std::exception {
protected:
    std::string msg;
public:
    explicit AppException(const std::string& m) : msg(m) {}
    virtual ~AppException() throw() {}
    const char* what() const throw() { return msg.c_str(); }
};
class InputException : public AppException {
public: explicit InputException(const std::string& m) : AppException("Ошибка ввода: " + m) {}
};
class IndexException : public AppException {
public: explicit IndexException(const std::string& m) : AppException("Ошибка индекса: " + m) {}
};
class FileException : public AppException {
public: explicit FileException(const std::string& m) : AppException("Ошибка файла: " + m) {}
};
class FormatException : public AppException {
public: explicit FormatException(const std::string& m) : AppException("Ошибка формата данных: " +
m) {}
};
#endif
```

4.2 Файл Utils.h

```
#ifndef UTILS_H
#define UTILS_H
#include <string>
#include <istream>

std::string readLine(const std::string& prompt);                    // ввод строки с клавиатуры
int readInt(const std::string& prompt, int min, int max);           // ввод целого в диапазоне
double readDouble(const std::string& prompt, double min, double max);
std::string getLineFrom(std::istream& in);                          // чтение строки из файла
int parseInt(const std::string& s);
double parseDouble(const std::string& s);
#endif
```

4.3 Файл Utils.cpp

```
#include "Utils.h"
#include "Exceptions.h"
#include <iostream>
#include <cstdlib>

using namespace std;

string getLineFrom(istream& in) {
    string s;
    if (!getline(in, s)) throw FormatException("неожиданный конец данных");
    if (!s.empty() && s[s.size() - 1] == '\r') s.erase(s.size() - 1);
    return s;
}
int parseInt(const string& s) {
```

```
    char* end = 0;
    long v = strtol(s.c_str(), &end, 10);
    if (s.empty() || *end != '\0') throw InputException("'" + s + "' не является целым числом");
    return (int)v;
}
double parseDouble(const string& s) {
    char* end = 0;
    string t = s;
    for (size_t i = 0; i < t.size(); ++i) if (t[i] == ',') t[i] = '.';
    double v = strtod(t.c_str(), &end);
    if (t.empty() || *end != '\0') throw InputException("'" + s + "' не является числом");
    return v;
}
string readLine(const string& prompt) {
    cout << prompt;
    string s;
    if (!getline(cin, s)) throw InputException("поток ввода закрыт");
    if (!s.empty() && s[s.size() - 1] == '\r') s.erase(s.size() - 1);
    if (s.empty()) throw InputException("пустая строка недопустима");
    return s;
}
int readInt(const string& prompt, int min, int max) {
    int v = parseInt(readLine(prompt));
    if (v < min || v > max) throw InputException("значение должно быть от " + to_string(min) + "
до " + to_string(max));
    return v;
}
double readDouble(const string& prompt, double min, double max) {
    double v = parseDouble(readLine(prompt));
    if (v < min || v > max) throw InputException("значение вне допустимого диапазона");
    return v;
}
```

4.4 Файл Base.h

```
#ifndef BASE_H
#define BASE_H
#include <string>
#include <iostream>

// Абстрактный базовый класс сотрудника/студента ВУЗа
class Base {
protected:
    std::string fio;
public:
    Base();
    explicit Base(const std::string& fio);
    Base(const Base& other);
    virtual ~Base();

    const std::string& getFio() const;
    void setFio(const std::string& f);

    virtual int type() const = 0;                       // 1 - студент, 2 - преподаватель, 3 -
администрация
    virtual const char* typeName() const = 0;
    virtual void print(std::ostream& os) const = 0;     // вывод на экран
    virtual void input() = 0;                           // ввод/изменение с клавиатуры
    virtual void save(std::ostream& os) const = 0;      // запись в файл
    virtual void load(std::istream& is) = 0;            // чтение из файла
    virtual Base* clone() const = 0;                    // копия объекта (для глубокого
копирования Keeper)
};
std::ostream& operator<<(std::ostream& os, const Base& b);
#endif
```

4.5 Файл Base.cpp

```
#include "Base.h"
#include "Exceptions.h"
using namespace std;

Base::Base() : fio("") { cout << "[Конструктор без параметров] Base" << endl; }
Base::Base(const string& f) : fio(f) { cout << "[Конструктор с параметрами] Base" << endl; }
Base::Base(const Base& o) : fio(o.fio) { cout << "[Конструктор копирования] Base" << endl; }
Base::~Base() { cout << "[Деструктор] Base" << endl; }
```

```
const string& Base::getFio() const { return fio; }
void Base::setFio(const string& f) {
    if (f.empty()) throw InputException("ФИО не может быть пустым");
    fio = f;
}
ostream& operator<<(ostream& os, const Base& b) { b.print(os); return os; }
```

4.6 Файл Student.h

```
#ifndef STUDENT_H
#define STUDENT_H
#include "Base.h"


class Student : public Base {
    std::string group, specialty;
    int course;
    double gpa;
public:
    Student();
    Student(const std::string& fio, const std::string& group, const std::string& specialty, int
course, double gpa);
    Student(const Student& o);
    ~Student();
    Student& operator=(const Student& o);
    bool operator==(const Student& o) const;

    const std::string& getGroup() const;     void setGroup(const std::string& g);
    const std::string& getSpecialty() const; void setSpecialty(const std::string& s);
    int getCourse() const;                   void setCourse(int c);
    double getGpa() const;                   void setGpa(double g);

    int type() const;
    const char* typeName() const;
    void print(std::ostream& os) const;
    void input();
    void save(std::ostream& os) const;
    void load(std::istream& is);
    Base* clone() const;
};
#endif
```

4.7 Файл Student.cpp

```
#include "Student.h"
#include "Exceptions.h"
#include "Utils.h"
using namespace std;


Student::Student() : Base(), group(""), specialty(""), course(1), gpa(0) {
    cout << "[Конструктор без параметров] Student" << endl;
}
Student::Student(const string& f, const string& g, const string& s, int c, double a) : Base(f) {
    setGroup(g); setSpecialty(s); setCourse(c); setGpa(a);
    cout << "[Конструктор с параметрами] Student" << endl;
}
Student::Student(const Student& o) : Base(o), group(o.group), specialty(o.specialty),
course(o.course), gpa(o.gpa) {
    cout << "[Конструктор копирования] Student" << endl;
}
Student::~Student() { cout << "[Деструктор] Student" << endl; }


Student& Student::operator=(const Student& o) {
    if (this != &o) { fio = o.fio; group = o.group; specialty = o.specialty; course = o.course;
gpa = o.gpa; }
    return *this;
}
bool Student::operator==(const Student& o) const {
    return fio == o.fio && group == o.group && specialty == o.specialty && course == o.course &&
gpa == o.gpa;
}


const string& Student::getGroup() const { return group; }
void Student::setGroup(const string& g) { if (g.empty()) throw InputException("группа не может
быть пустой"); group = g; }
const string& Student::getSpecialty() const { return specialty; }
void Student::setSpecialty(const string& s) { if (s.empty()) throw InputException("специальность
не может быть пустой"); specialty = s; }
```

```
int Student::getCourse() const { return course; }
void Student::setCourse(int c) { if (c < 1 || c > 6) throw InputException("курс должен быть от 1
до 6"); course = c; }
double Student::getGpa() const { return gpa; }
void Student::setGpa(double g) { if (g < 0 || g > 5) throw InputException("средний балл должен
быть от 0 до 5"); gpa = g; }


int Student::type() const { return 1; }
const char* Student::typeName() const { return "Студент"; }
void Student::print(ostream& os) const {
    os << "Студент: " << fio << ", группа " << group << ", специальность: " << specialty
       << ", курс " << course << ", средний балл " << gpa;
}
void Student::input() {
    setFio(readLine("ФИО: "));
    setGroup(readLine("Группа: "));
    setSpecialty(readLine("Специальность: "));
    setCourse(readInt("Курс (1-6): ", 1, 6));
    setGpa(readDouble("Средний балл (0-5): ", 0, 5));
}
void Student::save(ostream& os) const {
    os << fio << '\n' << group << '\n' << specialty << '\n' << course << '\n' << gpa << '\n';
}
void Student::load(istream& is) {
    setFio(getLineFrom(is)); setGroup(getLineFrom(is)); setSpecialty(getLineFrom(is));
    setCourse(parseInt(getLineFrom(is))); setGpa(parseDouble(getLineFrom(is)));
}
Base* Student::clone() const { return new Student(*this); }
```

4.8 Файл Teacher.h

```
#ifndef TEACHER_H
#define TEACHER_H
#include "Base.h"


class Teacher : public Base {
    std::string* groups;   int groupCount;     // динамический массив групп
    std::string* subjects; int subjectCount;   // динамический массив предметов
    static std::string* copyArr(const std::string* src, int n);
public:
    Teacher();
    Teacher(const std::string& fio, const std::string* groups, int gc, const std::string*
subjects, int sc);
    Teacher(const Teacher& o);
    ~Teacher();
    Teacher& operator=(const Teacher& o);
    bool operator==(const Teacher& o) const;


    int getGroupCount() const;    const std::string& getGroup(int i) const;
    int getSubjectCount() const;  const std::string& getSubject(int i) const;
    void setGroups(const std::string* g, int n);
    void setSubjects(const std::string* s, int n);
    void addGroup(const std::string& g);
    void addSubject(const std::string& s);


    int type() const;
    const char* typeName() const;
    void print(std::ostream& os) const;
    void input();
    void save(std::ostream& os) const;
    void load(std::istream& is);
    Base* clone() const;
};
#endif
```

4.9 Файл Teacher.cpp

```
#include "Teacher.h"
#include "Exceptions.h"
#include "Utils.h"
using namespace std;


string* Teacher::copyArr(const string* src, int n) {
    if (n <= 0) return 0;
    string* a = new string[n];
    for (int i = 0; i < n; ++i) a[i] = src[i];
    return a;
```

```
}
Teacher::Teacher() : Base(), groups(0), groupCount(0), subjects(0), subjectCount(0) {
    cout << "[Конструктор без параметров] Teacher" << endl;
}
Teacher::Teacher(const string& f, const string* g, int gc, const string* s, int sc)
    : Base(f), groups(0), groupCount(0), subjects(0), subjectCount(0) {
    setGroups(g, gc); setSubjects(s, sc);
    cout << "[Конструктор с параметрами] Teacher" << endl;
}
Teacher::Teacher(const Teacher& o) : Base(o), groups(copyArr(o.groups, o.groupCount)),
groupCount(o.groupCount),
    subjects(copyArr(o.subjects, o.subjectCount)), subjectCount(o.subjectCount) {
    cout << "[Конструктор копирования] Teacher" << endl;
}
Teacher::~Teacher() {
    delete[] groups; delete[] subjects;
    cout << "[Деструктор] Teacher" << endl;
}
Teacher& Teacher::operator=(const Teacher& o) {
    if (this != &o) {
        string* g = copyArr(o.groups, o.groupCount);
        string* s = copyArr(o.subjects, o.subjectCount);
        delete[] groups; delete[] subjects;
        fio = o.fio; groups = g; groupCount = o.groupCount; subjects = s; subjectCount =
o.subjectCount;
    }
    return *this;
}
bool Teacher::operator==(const Teacher& o) const {
    if (fio != o.fio || groupCount != o.groupCount || subjectCount != o.subjectCount) return
false;
    for (int i = 0; i < groupCount; ++i) if (groups[i] != o.groups[i]) return false;
    for (int i = 0; i < subjectCount; ++i) if (subjects[i] != o.subjects[i]) return false;
    return true;
}


int Teacher::getGroupCount() const { return groupCount; }
const string& Teacher::getGroup(int i) const {
    if (i < 0 || i >= groupCount) throw IndexException("нет группы с номером " + to_string(i));
    return groups[i];
}
int Teacher::getSubjectCount() const { return subjectCount; }
const string& Teacher::getSubject(int i) const {
    if (i < 0 || i >= subjectCount) throw IndexException("нет предмета с номером " +
to_string(i));
    return subjects[i];
}
void Teacher::setGroups(const string* g, int n) {
    if (n < 0) throw InputException("количество групп не может быть отрицательным");
    string* a = copyArr(g, n);
    delete[] groups; groups = a; groupCount = n;
}
void Teacher::setSubjects(const string* s, int n) {
    if (n < 0) throw InputException("количество предметов не может быть отрицательным");
    string* a = copyArr(s, n);
    delete[] subjects; subjects = a; subjectCount = n;
}
void Teacher::addGroup(const string& g) {
    if (g.empty()) throw InputException("группа не может быть пустой");
    string* a = new string[groupCount + 1];
    for (int i = 0; i < groupCount; ++i) a[i] = groups[i];
    a[groupCount] = g;
    delete[] groups; groups = a; ++groupCount;
}
void Teacher::addSubject(const string& s) {
    if (s.empty()) throw InputException("предмет не может быть пустым");
    string* a = new string[subjectCount + 1];
    for (int i = 0; i < subjectCount; ++i) a[i] = subjects[i];
    a[subjectCount] = s;
    delete[] subjects; subjects = a; ++subjectCount;
}


int Teacher::type() const { return 2; }
const char* Teacher::typeName() const { return "Преподаватель"; }
void Teacher::print(ostream& os) const {
    os << "Преподаватель: " << fio << "; группы: ";
    for (int i = 0; i < groupCount; ++i) os << (i ? ", " : "") << groups[i];
    os << "; предметы: ";
    for (int i = 0; i < subjectCount; ++i) os << (i ? ", " : "") << subjects[i];
```

```
}
void Teacher::input() {
    setFio(readLine("ФИО: "));
    setGroups(0, 0); setSubjects(0, 0);
    int n = readInt("Сколько групп ведёт (0-50): ", 0, 50);
    for (int i = 0; i < n; ++i) addGroup(readLine("  Группа " + to_string(i + 1) + ": "));
    n = readInt("Сколько предметов преподаёт (0-50): ", 0, 50);
    for (int i = 0; i < n; ++i) addSubject(readLine("  Предмет " + to_string(i + 1) + ": "));
}
void Teacher::save(ostream& os) const {
    os << fio << '\n' << groupCount << '\n';
    for (int i = 0; i < groupCount; ++i) os << groups[i] << '\n';
    os << subjectCount << '\n';
    for (int i = 0; i < subjectCount; ++i) os << subjects[i] << '\n';
}
void Teacher::load(istream& is) {
    setFio(getLineFrom(is));
    setGroups(0, 0); setSubjects(0, 0);
    int n = parseInt(getLineFrom(is));
    if (n < 0 || n > 1000) throw FormatException("некорректное количество групп");
    for (int i = 0; i < n; ++i) addGroup(getLineFrom(is));
    n = parseInt(getLineFrom(is));
    if (n < 0 || n > 1000) throw FormatException("некорректное количество предметов");
    for (int i = 0; i < n; ++i) addSubject(getLineFrom(is));
}
Base* Teacher::clone() const { return new Teacher(*this); }
```

4.10 Файл Admin.h

```
#ifndef ADMIN_H
#define ADMIN_H
#include "Base.h"


class Admin : public Base {
    std::string position, phone, area;
public:
    Admin();
    Admin(const std::string& fio, const std::string& position, const std::string& phone, const
std::string& area);
    Admin(const Admin& o);
    ~Admin();
    Admin& operator=(const Admin& o);
    bool operator==(const Admin& o) const;


    const std::string& getPosition() const; void setPosition(const std::string& p);
    const std::string& getPhone() const;    void setPhone(const std::string& p);
    const std::string& getArea() const;     void setArea(const std::string& a);


    int type() const;
    const char* typeName() const;
    void print(std::ostream& os) const;
    void input();
    void save(std::ostream& os) const;
    void load(std::istream& is);
    Base* clone() const;
};
#endif
```

4.11 Файл Admin.cpp

```
#include "Admin.h"
#include "Exceptions.h"
#include "Utils.h"
using namespace std;


Admin::Admin() : Base(), position(""), phone(""), area("") { cout << "[Конструктор без
параметров] Admin" << endl; }
Admin::Admin(const string& f, const string& pos, const string& ph, const string& a) : Base(f) {
    setPosition(pos); setPhone(ph); setArea(a);
    cout << "[Конструктор с параметрами] Admin" << endl;
}
Admin::Admin(const Admin& o) : Base(o), position(o.position), phone(o.phone), area(o.area) {
    cout << "[Конструктор копирования] Admin" << endl;
}
Admin::~Admin() { cout << "[Деструктор] Admin" << endl; }
Admin& Admin::operator=(const Admin& o) {
    if (this != &o) { fio = o.fio; position = o.position; phone = o.phone; area = o.area; }
    return *this;
```

```
}
bool Admin::operator==(const Admin& o) const {
    return fio == o.fio && position == o.position && phone == o.phone && area == o.area;
}

const string& Admin::getPosition() const { return position; }
void Admin::setPosition(const string& p) { if (p.empty()) throw InputException("должность не
может быть пустой"); position = p; }
const string& Admin::getPhone() const { return phone; }
void Admin::setPhone(const string& p) {
    if (p.empty()) throw InputException("телефон не может быть пустым");
    for (size_t i = 0; i < p.size(); ++i) {
        char c = p[i];
        if (!((c >= '0' && c <= '9') || c == '+' || c == '-' || c == ' ' || c == '(' || c ==
')'))
            throw InputException("телефон содержит недопустимые символы");
    }
    phone = p;
}
const string& Admin::getArea() const { return area; }
void Admin::setArea(const string& a) { if (a.empty()) throw InputException("область
ответственности не может быть пустой"); area = a; }

int Admin::type() const { return 3; }
const char* Admin::typeName() const { return "Административный персонал"; }
void Admin::print(ostream& os) const {
    os << "Администрация: " << fio << ", должность: " << position << ", телефон: " << phone
       << ", область ответственности: " << area;
}
void Admin::input() {
    setFio(readLine("ФИО: "));
    setPosition(readLine("Должность: "));
    setPhone(readLine("Телефон: "));
    setArea(readLine("Область ответственности: "));
}
void Admin::save(ostream& os) const {
    os << fio << '\n' << position << '\n' << phone << '\n' << area << '\n';
}
void Admin::load(istream& is) {
    setFio(getLineFrom(is)); setPosition(getLineFrom(is)); setPhone(getLineFrom(is));
setArea(getLineFrom(is));
}
Base* Admin::clone() const { return new Admin(*this); }
```

4.12 Файл Keeper.h

```
#ifndef KEEPER_H
#define KEEPER_H
#include "Base.h"
#include <string>

// Контейнер (ВУЗ): динамический массив указателей на производные объекты Base
class Keeper {
    Base** items;
    int count, capacity;
    void grow();
    void clear();
public:
    Keeper();
    Keeper(const Keeper& o);
    ~Keeper();
    Keeper& operator=(const Keeper& o);

    int size() const;
    void add(Base* obj);              // Keeper становится владельцем объекта
    void remove(int index);
    Base& operator[](int index);
    const Base& operator[](int index) const;
    void print(std::ostream& os) const;
    void saveToFile(const std::string& filename) const;
    void loadFromFile(const std::string& filename);
};
std::ostream& operator<<(std::ostream& os, const Keeper& k);
#endif
```

4.13 Файл Keeper.cpp

```
#include "Keeper.h"
```

```
#include "Student.h"
#include "Teacher.h"
#include "Admin.h"
#include "Exceptions.h"
#include "Utils.h"
#include <fstream>
using namespace std;

Keeper::Keeper() : items(0), count(0), capacity(0) { cout << "[Конструктор без параметров]
Keeper" << endl; }
Keeper::Keeper(const Keeper& o) : items(0), count(0), capacity(0) {
    cout << "[Конструктор копирования] Keeper" << endl;
    for (int i = 0; i < o.count; ++i) add(o.items[i]->clone());
}
Keeper::~Keeper() { clear(); cout << "[Деструктор] Keeper" << endl; }

Keeper& Keeper::operator=(const Keeper& o) {
    if (this != &o) {
        Keeper tmp(o);                          // копия; при исключении текущее состояние не
портится
        Base** p = items; items = tmp.items; tmp.items = p;
        int c = count; count = tmp.count; tmp.count = c;
        c = capacity; capacity = tmp.capacity; tmp.capacity = c;
    }
    return *this;
}
void Keeper::clear() {
    for (int i = 0; i < count; ++i) delete items[i];
    delete[] items;
    items = 0; count = capacity = 0;
}
void Keeper::grow() {
    int nc = capacity ? capacity * 2 : 4;
    Base** a = new Base*[nc];
    for (int i = 0; i < count; ++i) a[i] = items[i];
    delete[] items;
    items = a; capacity = nc;
}
int Keeper::size() const { return count; }
void Keeper::add(Base* obj) {
    if (!obj) throw InputException("нельзя добавить пустой указатель");
    if (count == capacity) grow();
    items[count++] = obj;
}
void Keeper::remove(int index) {
    if (index < 0 || index >= count) throw IndexException("нет элемента с номером " +
to_string(index + 1));
    delete items[index];
    for (int i = index; i < count - 1; ++i) items[i] = items[i + 1];
    --count;
}
Base& Keeper::operator[](int index) {
    if (index < 0 || index >= count) throw IndexException("нет элемента с номером " +
to_string(index + 1));
    return *items[index];
}
const Base& Keeper::operator[](int index) const {
    if (index < 0 || index >= count) throw IndexException("нет элемента с номером " +
to_string(index + 1));
    return *items[index];
}
void Keeper::print(ostream& os) const {
    if (count == 0) { os << "ВУЗ пуст." << endl; return; }
    for (int i = 0; i < count; ++i) os << (i + 1) << ") " << *items[i] << endl;
}
ostream& operator<<(ostream& os, const Keeper& k) { k.print(os); return os; }

void Keeper::saveToFile(const string& filename) const {
    ofstream out(filename.c_str());
    if (!out) throw FileException("не удалось открыть '" + filename + "' для записи");
    out << count << '\n';
    for (int i = 0; i < count; ++i) { out << items[i]->type() << '\n'; items[i]->save(out); }
    if (!out) throw FileException("ошибка записи в '" + filename + "'");
}
void Keeper::loadFromFile(const string& filename) {
    ifstream in(filename.c_str());
    if (!in) throw FileException("не удалось открыть '" + filename + "' для чтения");
    Keeper tmp;                                 // читаем во временный контейнер
```

```
    int n = parseInt(getLineFrom(in));
    if (n < 0) throw FormatException("отрицательное количество записей");
    for (int i = 0; i < n; ++i) {
        int t = parseInt(getLineFrom(in));
        Base* obj = 0;
        if (t == 1) obj = new Student();
        else if (t == 2) obj = new Teacher();
        else if (t == 3) obj = new Admin();
        else throw FormatException("неизвестный тип записи: " + to_string(t));
        try { obj->load(in); tmp.add(obj); }
        catch (...) { delete obj; throw; }
    }
    Base** p = items; items = tmp.items; tmp.items = p;   // обмен: tmp удалит старое содержимое
    int c = count; count = tmp.count; tmp.count = c;
    c = capacity; capacity = tmp.capacity; tmp.capacity = c;
}
```

4.14 Файл main.cpp

```
#include "Keeper.h"
#include "Student.h"
#include "Teacher.h"
#include "Admin.h"
#include "Exceptions.h"
#include "Utils.h"
#include <iostream>
using namespace std;


static void menu() {
    cout << "\n===== ВУЗ =====\n"
         << "1. Добавить студента\n2. Добавить преподавателя\n3. Добавить администратора\n"
         << "4. Показать все\n5. Показать одного по номеру\n6. Изменить по номеру\n"
         << "7. Удалить по номеру\n8. Сохранить в файл\n9. Загрузить из файла\n0. Выход\n";
}
static void add(Keeper& k, Base* obj) {
    try { obj->input(); k.add(obj); cout << "Добавлено." << endl; }
    catch (...) { delete obj; throw; }
}
int main() {
    Keeper vuz;
    bool run = true;
    while (run) {
        try {
            menu();
            int c = readInt("Выбор: ", 0, 9);
            switch (c) {
                case 1: add(vuz, new Student()); break;
                case 2: add(vuz, new Teacher()); break;
                case 3: add(vuz, new Admin()); break;
                case 4: cout << vuz; break;
                case 5: { int i = readInt("Номер: ", 1, 1000000); cout << vuz[i - 1] << endl;
break; }
                case 6: { int i = readInt("Номер: ", 1, 1000000); cout << vuz[i - 1] << endl;
vuz[i - 1].input(); cout << "Изменено." << endl; break; }
                case 7: { int i = readInt("Номер: ", 1, 1000000); vuz.remove(i - 1); cout <<
"Удалено." << endl; break; }
                case 8: vuz.saveToFile(readLine("Имя файла: ")); cout << "Сохранено." << endl;
break;
                case 9: vuz.loadFromFile(readLine("Имя файла: ")); cout << "Загружено." << endl;
break;
                case 0: run = false; break;
            }
        } catch (const AppException& e) {
            cout << e.what() << endl;
            if (cin.eof()) break;
        }
    }
    return 0;
}
```

5 Результаты работы программы

Программа собирается командой clang++ -std=c++11 -Wall -Wextra -o lab1 *.cpp (или g++ с теми же параметрами) без предупреждений. Ниже приведены примеры работы с описанием действий пользователя и ответов программы.

5.1 Добавление студента

Пользователь выбирает пункт 1 главного меню. Программа создает объект Student (выводятся сообщения конструкторов Base и Student) и по очереди запрашивает поля. После ввода всех данных объект передается контейнеру.

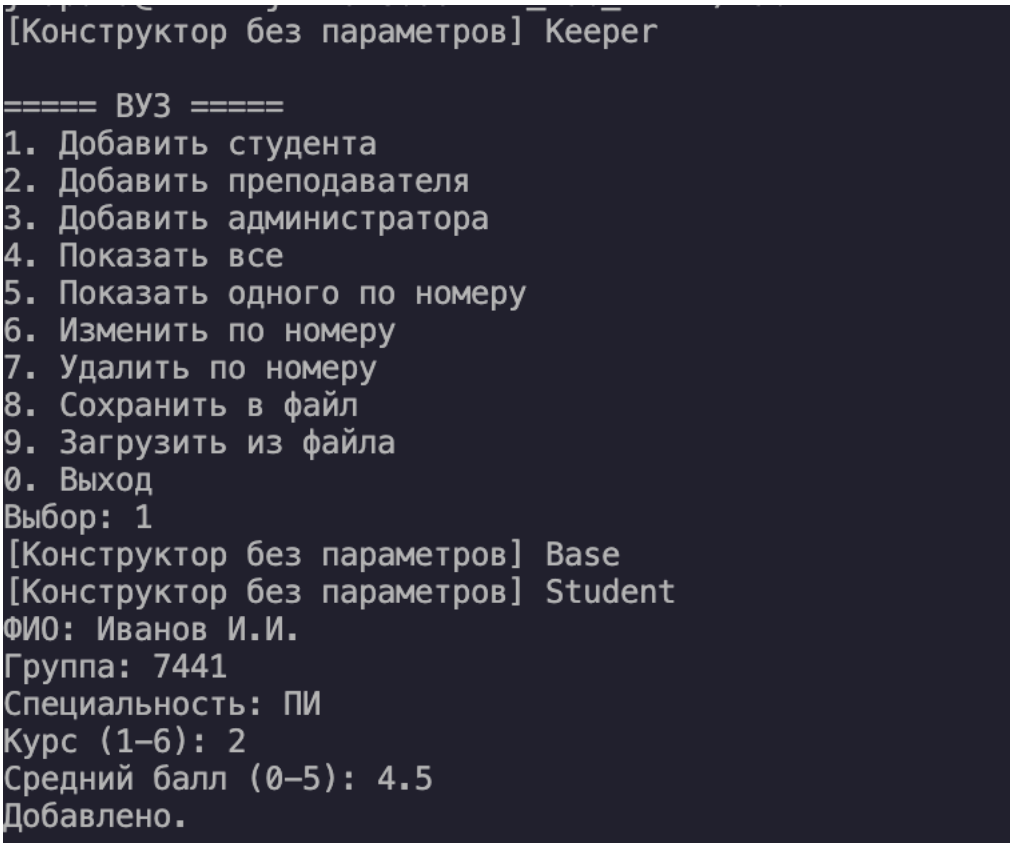


Рисунок 2 — Добавление студента (пункт 1 меню)

На рисунке 2 показана последовательность запросов при вводе студента. Аналогично работают пункты 2 (преподаватель: после ФИО запрашиваются количество и названия групп, затем количество и названия предметов) и 3 (администрация: должность, телефон, область ответственности).

5.2 Просмотр содержимого

При выборе пункта 4 программа выводит все объекты контейнера с порядковыми номерами. Для каждого объекта вызывается свой метод print (полиморфизм).

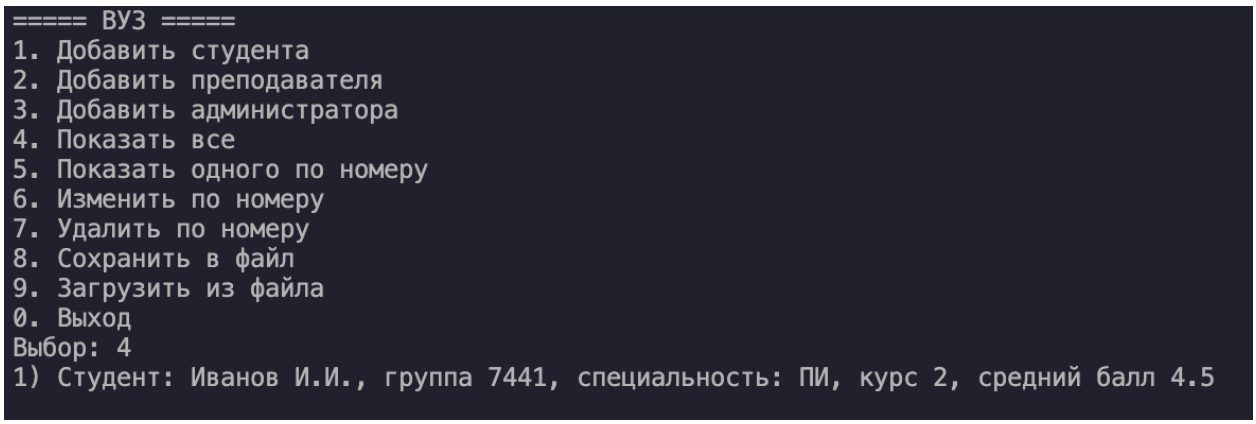


Рисунок 3 — Вывод всех объектов (пункт 4 меню)

На рисунке 3 видно, что в одном контейнере хранятся объекты трех разных классов, и каждый выводится в своем формате.

5.3 Сохранение и загрузка

При выборе пункта 8 пользователь вводит имя файла, и контейнер полностью записывается в него. Пункт 9 восстанавливает контейнер из файла. Для трех объектов из предыдущего примера файл имеет следующее содержимое:

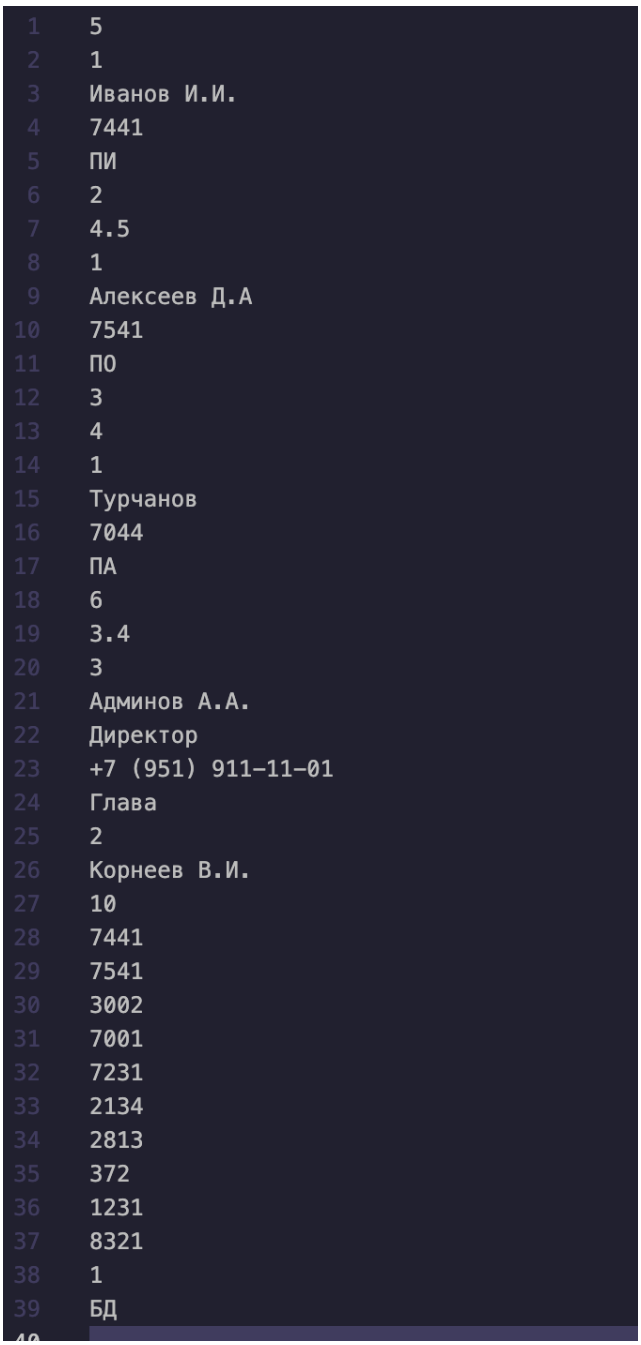


Рисунок 4 — Содержимое файла сохранения

На рисунке 4 первая строка содержит количество объектов (3), далее для каждого объекта записан код типа и его поля. Для преподавателя после ФИО указаны количество групп (2), сами группы, количество предметов (1) и предмет.

5.4 Обработка исключительных ситуаций

При некорректных действиях пользователя программа выводит поясняющий текст исключения и возвращается в главное меню, не завершаясь аварийно.

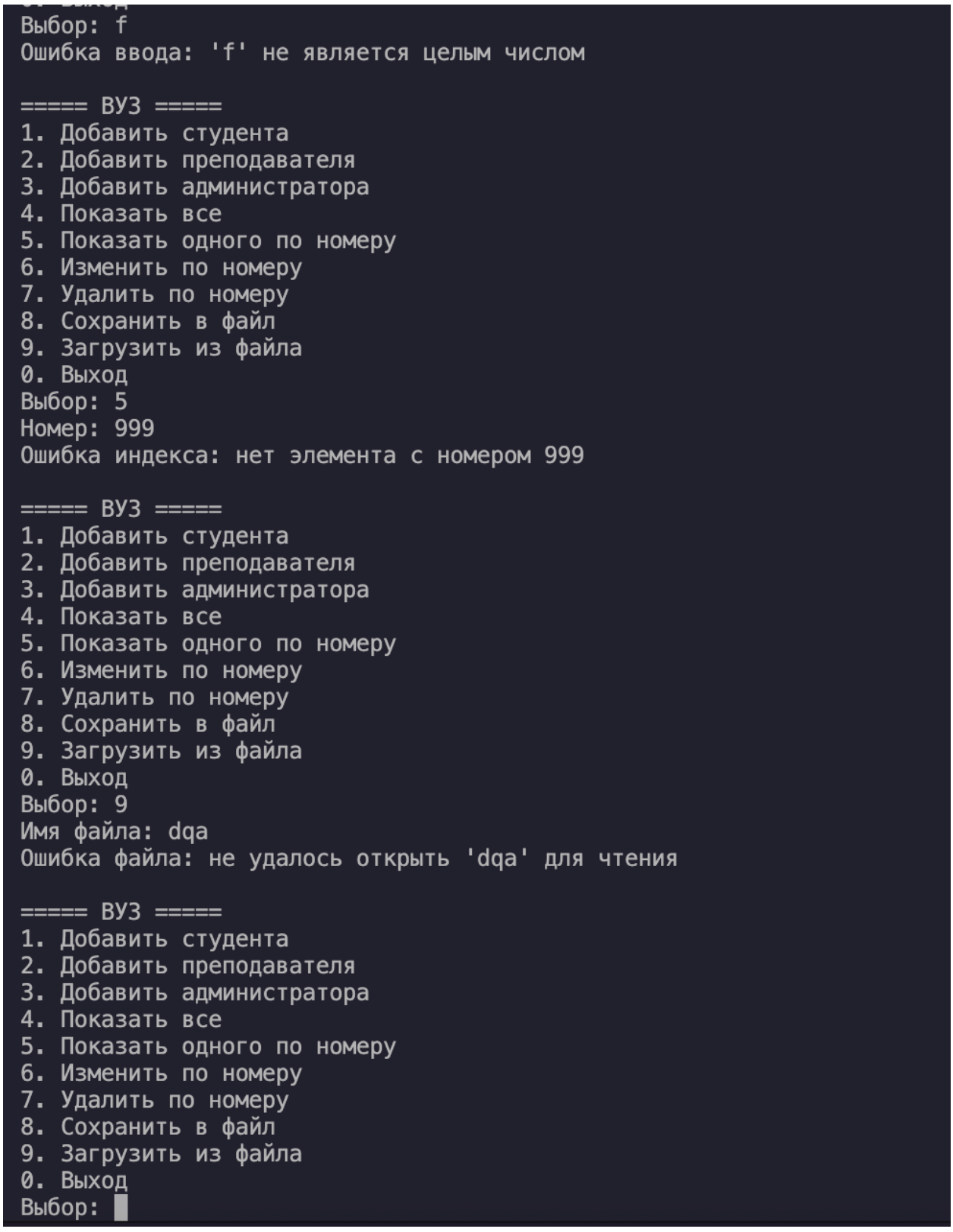


Рисунок 5 — Сообщения об исключительных ситуациях

На рисунке 5 показаны три вида ошибок: нечисловой ввод в меню, обращение к несуществующему элементу и открытие несуществующего файла.

5.5 Вызовы конструкторов и деструкторов

Каждый вызов конструктора и деструктора сопровождается сообщением. При удалении студента (пункт 7) выводятся сообщения деструкторов в порядке «производный класс, затем базовый», а при выходе из программы — деструкторы всех оставшихся объектов и контейнера.

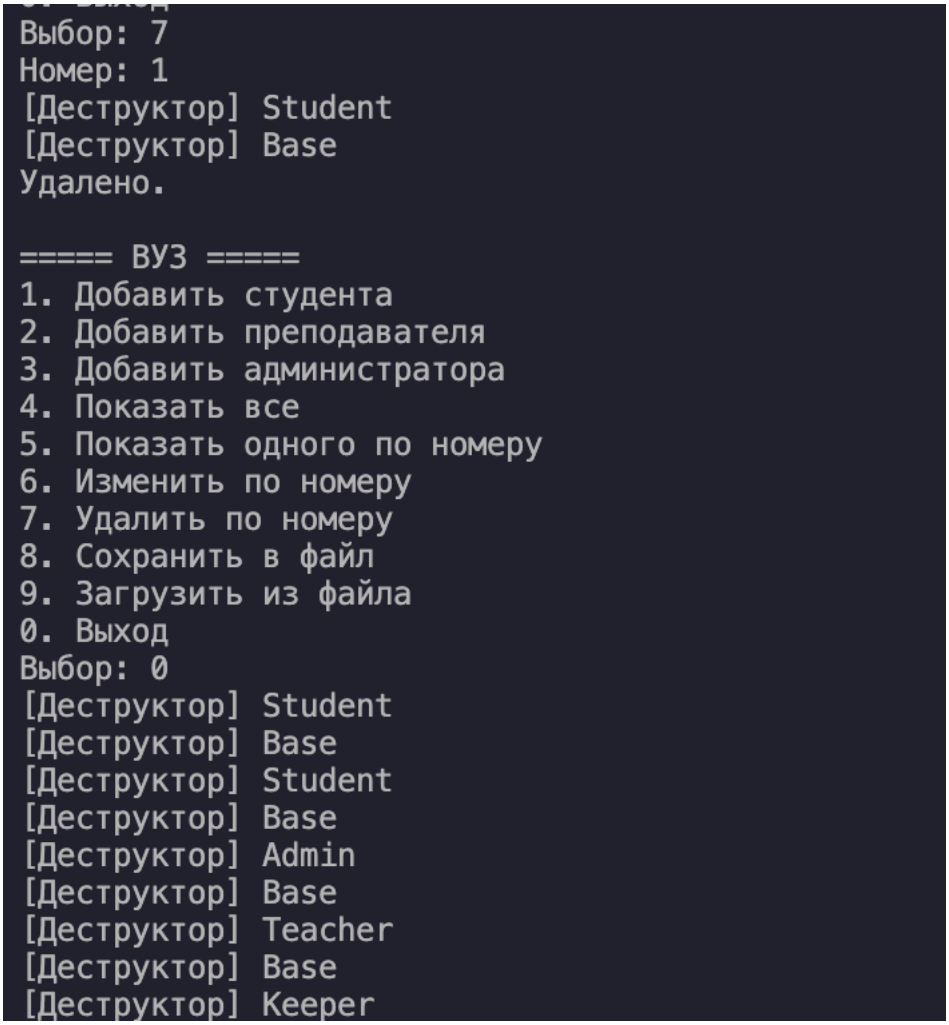


Рисунок 6 — Сообщения деструкторов

На рисунке 6 показан порядок освобождения: сначала деструктор производного класса, затем деструктор базового класса, после всех элементов — деструктор Keeper.

6 Выводы

В ходе лабораторной работы разработан класс-контейнер Keeper, который хранит объекты производных классов Student, Teacher и Admin абстрактного класса Base через указатели и обеспечивает добавление, удаление, просмотр, изменение, а также полное сохранение в файл и восстановление из файла.

На практике изучены и применены: наследование и абстрактные классы; виртуальные функции и полиморфизм при работе с объектами через указатель на базовый класс; динамическое выделение памяти (массив указателей в контейнере с удвоением емкости, динамические массивы групп и предметов преподавателя); правило трех (конструктор копирования, оператор присваивания, деструктор); перегрузка операторов (<<, ==, =, []); собственная иерархия исключений с поясняющими сообщениями; разделение программы на заголовочные файлы и файлы реализации.

Загрузка из файла выполняется во временный контейнер, поэтому поврежденный файл не уничтожает текущие данные, а все ошибки ввода и работы с файлами обрабатываются исключениями без аварийного завершения программы.